

ARK-455 Preregistration

ProofRecord Verification After Tampering

Series: ExecutionProof ARK Authorization-Boundary Track

Version: v1.0 — locked as `ark-455-v1.0-lock` immediately before execution

Repository: <https://github.com/derekhone/executionproof-testbeds>

Folder: `ark-455/`

Target tags: `ark-455-v1.0-lock` (preregistration LOCK) → `ark-455-v1.0` (execution result)

Prepared for: Derek Hone, Remnant Fieldworks Inc.

Date drafted: 2026-07-17

Lock date: To be set at moment of code hash-lock

LOCK INSTRUCTION

Before any test data is generated, the following steps must be completed in strict order:

1. Finalize this document. No further changes after step 2.
2. Compute SHA-256 hash of this file, the ProofRecord schema, and all verifier code files.
3. Commit the MANIFEST (see Section 16) to `executionproof-testbeds`.
4. Push and tag the commit `ark-455-v1.0-lock`.
5. Run the kill-gate calibration (Section 6). Record the pass/fail outcome **before any arm execution**.
6. Execute all arms with recorded seeds (Section 8). Record all verification outcomes **before computing any verdict**.
7. Compute verdict only after all arm results are recorded.

No criterion may be changed, loosened, or added after step 2. A FAIL under these criteria is a valid, publishable result. A KILLED verdict at the kill-gate is a valid, publishable result. The value of the protocol is that it constrains these choices before any data is seen.

1. Preamble and Series Context

1.1 Where This Fits

The ARK series through ARK-452 established the ExecutionProof authorization boundary across eight distinct dimensions:

Experiment	Dimension Established
ARK-441 / ARK-446	Boundary exists and replicates across devices
ARK-442	Temporal: stale authority fails closed
ARK-444	Integrity: altered decision record fails closed
ARK-443	Multi-party: single compromised authorizer cannot bypass quorum
ARK-445 / ARK-445b	Three-state: HOLD is a metrologically separable first-class outcome
ARK-447	Noise: Pauli twirling provides modest, measurable improvement
ARK-448	Protocol: gate-stop halts before spending budget on marginal conditions
ARK-449	State change: authorization currency expires with world state
ARK-452	Workflow: prior valid steps do not confer inherited authorization

What the corpus has not yet tested: **whether an independent verifier can reliably detect tampering in a ProofRecord and reject every altered record while accepting the original.**

This is the gap ARK-455 closes. It is the first experiment to validate the **commercial ProofRecord artifact** that customers would actually inspect.

1.2 The Doctrine Being Tested

The central ExecutionProof claim about audit integrity is:

An independent verifier with access to the original signature and schema can detect every tampering attempt and reject the altered record.

A system that produces authorization records without independent verifiability is a walled garden — the issuer’s claims are not externally auditable. ExecutionProof holds that auditability is non-negotiable: every ProofRecord must be independently verifiable, and every tampering attempt must be detectably invalid.

ARK-455 is the direct experimental test of that doctrine. It is not a quantum noise study. It is not a hardware characterization. It is a test of whether the complete ProofRecord verification model defeats the tampering attacks that produce compliance failures in real-world audit scenarios.

1.3 Precise Differentiation from ARK-444

ARK-444 tested **pre-execution tamper detection**: an authorization decision was altered before execution, and the system had to reject the execution attempt. The verifier was internal to the execution boundary.

ARK-455 tests **post-execution independent verification**: an authorization record from a completed execution is presented to an external auditor, and the auditor must detect whether any field was altered after issuance. The verifier is external to the execution boundary.

Dimension	ARK-444	ARK-455
Verifier position	Internal — within execution boundary	External — independent auditor
Timing	Pre-execution tamper detection	Post-execution record verification
Substrate	Quantum circuit (IBM hardware)	Classical software (local execution)
Question	“Can the system block tampered execution?”	“Can an auditor detect tampered records?”
Doctrine tested	Pre-execution integrity	Post-execution auditability

These are different questions with different commercial implications. ARK-444 demonstrated that the execution boundary rejects tampered decisions. ARK-455 demonstrates that external auditors can verify record integrity after issuance.

2. Primary Hypothesis

H1 (Primary): An independent verifier, given the ProofRecord schema, signature verification key, and a set of records where exactly one field has been altered post-issuance, will:

- **REJECT** every tampered record (all 7 tampering targets)
- **ACCEPT** the original unaltered record

Operationally:

- **ACCEPT condition:** Original record with valid signature $\rightarrow V_accept_rate \geq 0.95$
- **REJECT condition:** Tampered record (any field altered) with invalid signature $\rightarrow V_reject_rate \geq 0.95$

H1 is confirmed if and only if **all three primary pass criteria** in Section 9 are met simultaneously.

3. Secondary Hypotheses

H2a — Tampering universality: The type of field tampered is irrelevant to the REJECT outcome. Whether decision, timestamp, payload_hash, evidence_references, actor identity, execution_outcome, or review_path is altered — every tampering target produces $V_reject \geq 0.95$. No individual tampering target fails to be detected.

H2b — Dual verifier agreement: Two independent verifiers (V1 in TypeScript, V2 in Python) built from the same prose specification but with no shared source code agree on verification outcomes with

≥ 99% concordance on 100 non-adversarial test pairs during kill-gate calibration. This establishes that the verification logic is deterministic and reproducible across independent implementations.

H2c — Signature binding: Altering any field breaks the Ed25519 signature, and no tampered record in any arm produces a valid signature. Signature verification is the sole rejection mechanism — there is no ad-hoc field-validation heuristic that masks signature failures.

4. Record Architecture

4.1 ProofRecord Schema

The ProofRecord is a JSON object with seven required fields, canonicalized via RFC 8785 (JSON Canonicalization Scheme, JCS) before signing:

```
{
  "decision": "ALLOW | DENY | HOLD",
  "timestamp": "ISO 8601 UTC string",
  "payload_hash": "SHA-256 hex string of the payload",
  "evidence_references": ["array", "of", "evidence", "URIs"],
  "actor": "identity string of the decision-making authority",
  "execution_outcome": "executed | blocked | held",
  "review_path": "audit trail identifier"
}
```

4.2 Signature Generation

1. Canonicalize the record via RFC 8785 (JCS) → deterministic byte string
2. Sign the canonical byte string with Ed25519 private key → 64-byte signature
3. Append signature to the record as `"signature": "hex string"`

The signed record is:

```
{
  "decision": "...",
  "timestamp": "...",
  "...",
  "signature": "ed25519_signature_hex"
}
```

4.3 Verification Logic

An independent verifier receives:

- The signed ProofRecord (JSON with signature field)
- The Ed25519 public key (separate, not embedded in the record)
- The ProofRecord schema specification (this document)

Verification steps:

1. Extract the signature field from the record
2. Remove the signature field, leaving the original 7 fields
3. Canonicalize the 7-field record via RFC 8785 (JCS)
4. Verify the signature against the canonical byte string using the public key
5. **ACCEPT** if signature is valid; **REJECT** if invalid

No field-level validation beyond signature verification is performed. The verifier does not check whether the timestamp is well-formed, whether the payload_hash matches any payload, or whether the evidence_references are reachable. Signature validity is the sole criterion.

5. Arm Specifications

Eight arms. 100 test records per arm. All eight arms executed in sequence with recorded seeds.

Arm	Label	Tampering Target	Expected Outcome	Criteria
1	ACCEPT-original	None — original valid record	$V_{\text{accept}} \geq 0.95$	H1, H2c
2	REJECT-decision	decision field altered	$V_{\text{reject}} \geq 0.95$	H1, H2a
3	REJECT-timestamp	timestamp field altered	$V_{\text{reject}} \geq 0.95$	H1, H2a
4	REJECT-payload_hash	payload_hash field altered	$V_{\text{reject}} \geq 0.95$	H1, H2a
5	REJECT-evidence_refs	evidence_references array altered	$V_{\text{reject}} \geq 0.95$	H1, H2a
6	REJECT-actor	actor identity altered	$V_{\text{reject}} \geq 0.95$	H1, H2a
7	REJECT-outcome	execution_outcome altered	$V_{\text{reject}} \geq 0.95$	H1, H2a
8	REJECT-review_path	review_path altered	$V_{\text{reject}} \geq 0.95$	H1, H2a

5.1 Record Generation Procedure

For each arm:

- Generate seed:** Use a cryptographically secure random source, record the seed value before any record generation
- Generate base record:** All 7 fields filled with valid-looking but non-production data (decision = "ALLOW" | "DENY" | "HOLD", timestamp = ISO 8601, payload_hash = SHA-256 of random bytes, evidence_references = ["urn:evidence:001", "urn:evidence:002"], actor = "system:authorizer:001", execution_outcome = "executed" | "blocked" | "held", review_path = "audit:trace:NNNN")
- Sign the base record:** Canonicalize via JCS, sign with Ed25519 private key, append signature
- For Arm 1 (ACCEPT-original):** Keep the record unaltered

5. **For Arms 2-8 (REJECT-*)**: Alter the specified field (e.g., change decision “ALLOW” → “DENY”, increment timestamp by 1 second, flip one bit in payload_hash, append an element to evidence_references, change actor string, change execution_outcome, change review_path). **Do NOT re-sign**. The altered record retains the original signature, which is now invalid.

5.2 Note on Tampering Realism

All tamper operations produce records that are **structurally well-formed JSON** but cryptographically invalid. An auditor without the verification key cannot distinguish a tampered record from a valid one by inspecting field values alone — the signature is the only detection mechanism. This models the real-world scenario where an attacker with write access to a record store alters a field value but cannot forge a valid signature.

6. Kill-Gate Calibration

The kill-gate runs before any arm execution. If the kill-gate fails, arm execution is **not performed**. The experiment is recorded as **KILLED AT CALIBRATION GATE**. No threshold loosening, no re-running until it passes, no proceeding on a failed gate. These prohibitions are absolute under the lock.

6.1 Calibration Procedure

Generate 100 non-adversarial test pairs:

- 50 valid signed records (no tampering)
- 50 tampered records (random field alteration, no re-signing)

Run both verifiers (V1 and V2) on all 100 records. Record each verifier’s decision (ACCEPT | REJECT) for each record.

6.2 Kill-Gate Pass Criteria

Both checks must pass. Either failing → KILLED.

Check	Threshold	Pass Condition
Concordance	$\geq 99\%$	V1 and V2 agree on ≥ 99 of 100 test records
Sanity	100% on valid records	Both verifiers ACCEPT all 50 valid records

Concordance establishes that the two independent implementations produce the same verification logic. **Sanity** establishes that neither verifier is pathologically rejecting valid records.

If concordance < 99%: the two verifiers disagree on fundamental verification logic → KILLED. Diagnostic: identify the divergent record(s), inspect both implementations, publish the divergence as a finding.

If sanity fails (either verifier rejects a valid record): the verifier implementation is broken → KILLED. Do not proceed to arms.

6.3 Kill-Gate Role

The kill-gate is a **gating diagnostic only**. It validates that the two independent verifiers are functionally equivalent before any experimental arms are executed. A kill-gate failure is a valid, publishable

result — it means the dual-verifier architecture revealed an implementation divergence before any tampering claims were made.

7. Verifier Independence Rule

V1 (TypeScript) and **V2 (Python)** are built independently from the prose specification in this preregistration document. The V2 implementation:

- **Must NOT read or reference** any V1 source code
- **Must NOT read or reference** any generator source code
- **Must be built solely** from Sections 4.2, 4.3, and 6 of this document, which specify the canonicalization algorithm (RFC 8785), the signature algorithm (Ed25519), and the verification procedure

The V1 implementation is written first. The V2 implementation is written afterward, in a separate session, with no access to V1 or generator source files. Both implementations are committed and hash-locked before any test data is generated.

This isolation ensures that the verification logic is reproducible from the prose specification alone — a critical requirement for external auditors who will implement their own verifiers in production.

8. Execution Plan

Phase	Activity	Output
1. Lock	Commit MANIFEST (hashes of this document + schema + all verifier code)	Tag: <code>ark-455-v1.0-lock</code>
2. Kill-gate	Run 100-pair calibration (V1 vs V2 concordance + sanity)	Pass → continue; Fail → KILLED
3. Arm execution	Generate and verify 100 records per arm (8 arms × 100 = 800 records)	All outcomes recorded with seeds
4. Verdict	Compute PASS / FAIL / KILLED from recorded outcomes	RESULTS.md

8.1 Strict Execution Sequence

All steps must be executed in order. Commit timestamps must prove the ordering.

1. Commit MANIFEST (SHA-256 hashes of this document + schema + verifier code)
→ push → tag ark-455-v1.0-lock
2. Run kill-gate calibration per Section 6
→ record concordance % and sanity results in execution log
→ commit execution log entry
- 3a. If kill-gate PASSED → proceed to step 4
- 3b. If kill-gate FAILED → record KILLED verdict → stop. No arms executed.
4. For each arm (1–8):
 - a. Generate seed, record in execution log, commit
 - b. Generate 100 records per Section 5, record generation parameters
 - c. Run V1 on all 100 records, record outcomes (ACCEPT | REJECT per record)
 - d. Run V2 on all 100 records, record outcomes
 - e. Commit all outcomes before moving to next arm
5. After all arms complete → compute verdict per Section 11 → write RESULTS.md

9. Pass / Fail / Killed Criteria (Primary — Preregistered)

Three possible verdicts: **PASS**, **FAIL**, or **KILLED**.

9.1 KILLED Verdict

The experiment is **KILLED** if:

- **Kill-gate fails** (concordance < 99% OR sanity check fails)

A KILLED verdict is assigned before any arm execution. It is a valid, publishable result. It means the dual-verifier architecture revealed a blocker before any tampering claims were made.

9.2 PASS Verdict

The experiment **PASSES** if and only if **all three** of the following criteria are met simultaneously:

#	Criterion	Threshold	Arms
C1	V_accept_original — verification acceptance rate on original unaltered records	≥ 0.95	Arm 1
C2	V_reject_min — minimum rejection rate across all tampering targets	≥ 0.95	Arms 2–8
C3	V_margin = V_reject_min – (1 – V_accept_original) — decision margin	≥ 0.85	Derived

Interpretation:

- **C1:** At least 95 of 100 original valid records are accepted by both verifiers
- **C2:** For each of the 7 tampering targets, at least 95 of 100 tampered records are rejected by both verifiers
- **C3:** The decision boundary is separated by at least 0.85 (e.g., $V_{\text{accept}} = 0.97$, $V_{\text{reject_min}} = 0.96 \rightarrow \text{margin} = 0.96 - 0.03 = 0.93 \geq 0.85$)

9.3 FAIL Verdict

The experiment **FAILS** if:

- Kill-gate passes (no KILLED verdict), AND
- Any of C1, C2, or C3 is not satisfied

9.4 Verifier Agreement Requirement

For all metrics (C1, C2, C3), the reported value is the **worst case** between V1 and V2. If V1 accepts 97% of original records and V2 accepts 96%, $C1 = 0.96$. This ensures that both verifiers independently meet the criteria — the experiment does not pass if only one verifier succeeds.

10. Secondary Metrics (Descriptive — Not Pass/Fail)

Reported regardless of primary verdict. These provide diagnostic depth and inform the forward roadmap. They do not alter the pass/fail outcome.

Metric	Description	Hypothesis
V_reject per arm	Rejection rate for each tampering target (Arms 2-8) individually	H2a
V_accept_V1 vs V_accept_V2	Per-verifier acceptance rates on Arm 1	H2b
Concordance_per_arm	V1-V2 agreement % for each arm	H2b
Signature_validity_check	Confirm that zero tampered records have valid signatures	H2c

If any individual tampering target shows $V_{\text{reject}} < 0.95$ while others succeed, the secondary analysis identifies which field tampering was not reliably detected — a finding worth publishing precisely because it names the schema or signature constraint.

11. Analysis Plan

11.1 Raw Outcome Extraction

For each arm, for each verifier (V1, V2), extract:

- `n_accept` — number of records the verifier accepted (signature valid)
- `n_reject` — number of records the verifier rejected (signature invalid)
- `n_total` = 100 (fixed per arm)
- `V_accept` = `n_accept` / `n_total`
- `V_reject` = `n_reject` / `n_total`

11.2 Primary Metric Computation

```
# Arm 1 (original valid records)
V_accept_V1 = n_accept_V1(arm=1) / 100
V_accept_V2 = n_accept_V2(arm=1) / 100
V_accept_original = min(V_accept_V1, V_accept_V2) # worst case

# Arms 2-8 (tampered records)
V_reject_per_arm = []
for arm in [2, 3, 4, 5, 6, 7, 8]:
    V_reject_V1 = n_reject_V1(arm) / 100
    V_reject_V2 = n_reject_V2(arm) / 100
    V_reject_arm = min(V_reject_V1, V_reject_V2) # worst case
    V_reject_per_arm.append(V_reject_arm)

V_reject_min = min(V_reject_per_arm)

# Decision margin
V_margin = V_reject_min - (1 - V_accept_original)
```

11.3 Verdict Assignment

```
if not kill_gate_passed:
    VERDICT = "KILLED AT CALIBRATION GATE"
elif V_accept_original >= 0.95 and V_reject_min >= 0.95 and V_margin >= 0.85:
    VERDICT = "PASS"
else:
    VERDICT = "FAIL"
```

11.4 Secondary Analysis

- Report `V_reject` per arm to evaluate H2a (tampering universality — no single field fails detection)
- Report `V_accept_V1` vs `V_accept_V2` and per-arm concordance to evaluate H2b (dual verifier agreement)
- Verify that zero tampered records have valid signatures to confirm H2c (signature binding)
- If FAIL: identify which criterion failed and which arm(s) contributed — this is the diagnostic for the next experiment

12. What Constitutes Failure (and Why Each Is Still Valuable)

12.1 Any tampering arm shows $V_{\text{reject}} < 0.95$

A specific field tampering is not reliably detected. This is a significant, publishable finding: it names the exact field or signature mechanism that the current verification logic does not handle. The secondary analysis identifies which arm and at what level, giving the forward program a precise target.

12.2 Original records show $V_{\text{accept}} < 0.95$

Valid records are being rejected. This would indicate that the verification logic is either too restrictive, or the signature mechanism is producing false negatives. Diagnostic: inspect the rejected records, verify signature generation, check JCS canonicalization implementation.

12.3 $V_{\text{margin}} < 0.85$

The decision boundary exists in principle but without sufficient separation to be practically meaningful. The two outcome states (accept vs reject) are not reliably distinguishable at scale. This would not contradict H1 on its own but would substantially limit the claim about practical deployability.

12.4 Kill-gate fails

The two independent verifiers disagree on verification logic (concordance $< 99\%$) or one rejects valid records (sanity check fails). No verdict on tampering detection is claimed. The experiment is recorded as a preregistered KILLED. A future attempt requires debugging the divergence, publishing the finding, and locking a new run — not a silent retry of this one.

All three outcomes (PASS, FAIL, KILLED) are valid, publishable results under the preregistration-first protocol. A FAIL that identifies which field tampering is missed is worth more to the program than a marginal pass that obscures the detection boundary.

13. Pre-Execution Commitments (Lock Rules)

The following are unconditionally prohibited after the MANIFEST is committed (step 1 of Section 8):

1. **No criterion changes.** Thresholds, metric definitions, and the pass/fail/killed verdict logic may not be altered.
2. **No arm additions or removals.** The eight-arm design is fixed. Arms may not be excluded from the primary verdict calculation after data is seen.
3. **No rescue-after-kill.** If the kill-gate fails: no re-running until it passes, no threshold loosening, no soft kill that proceeds to arms anyway.
4. **No post-hoc subgroup selection.** If some tampering arms pass and others fail, the verdict is FAIL — not “partial pass excluding arm N.”
5. **No silent technical corrections after data is recorded.** Any technical error discovered before data is recorded (generator bug, verifier crash, seed collision) is documented and corrected in a pre-data correction note. Any correction after data is recorded requires full explicit disclosure and tagging.
6. **Verifier code is committed at lock.** The verifier implementations that produce the verdict run on the committed test data without post-hoc modification.

7. **Generator code is committed at lock.** The record generator and tampering logic are fixed before any test data is created.
8. **Seeds are recorded before generation.** Every arm's random seed is committed to the execution log before any records for that arm are generated.

14. Honest Boundary Statements

These belong in every external conversation that references ARK-455 results.

1. Classical software experiment. ARK-455 is fully classical and runs locally. No IBM quantum hardware is involved, no QPU budget is consumed. The “execution” is the running of two software verifiers on locally generated test records.

2. Non-production test data. All ProofRecord test data is synthetically generated with random field values and does not represent real authorization decisions, real payloads, or real evidence. The experiment tests the verification logic, not the generation or issuance workflow.

3. Single-field tampering only. Each tampering arm alters exactly one field. Multi-field tampering (e.g., altering both decision and timestamp) is not tested. The experiment validates that single-field alterations are detected; it does not claim to enumerate all possible tampering strategies.

4. No adversarial signature attacks. The experiment tests tampering by altering record fields without re-signing. It does not test signature forgery, key compromise, collision attacks, or side-channel attacks on the Ed25519 implementation. Those are deferred to future cryptographic-focused experiments.

5. Verifier independence is procedural, not formal. V2 is built from the prose spec without reading V1 source code, following a manual isolation discipline. This is not a formal proof of independent derivation — it is a best-effort procedural control to ensure the verification logic is reproducible from the spec.

6. Schema simplicity. The ProofRecord schema has 7 fields and is deliberately minimal for this first verification experiment. Production ProofRecords may have additional fields, nested structures, or richer evidence references. The experiment validates the core signature-binding mechanism on a representative schema.

7. Not a security audit. This is a functional test of the verification logic under preregistered tampering scenarios. It is not a formal security audit of the Ed25519 library, the JCS implementation, or the JSON parsing logic. Those are assumed correct per their respective specifications.

15. Connection to Corpus and Forward Roadmap

15.1 What ARK-455 Adds to the Established Corpus

If ARK-455 passes, the corpus establishes:

Dimension	Established By
Boundary exists and replicates across devices	ARK-441, ARK-446
Temporal: stale authority fails closed	ARK-442
Integrity: tampered action fails closed	ARK-444
Multi-party: single compromised authorizer fails closed	ARK-443
Three-state: HOLD is metrologically separable	ARK-445b
Noise: empirically testable mitigation	ARK-447
Protocol: gate-stop preserves integrity under marginal conditions	ARK-448
State change: authorization currency expires with world state	ARK-449
Workflow: prior valid steps do not confer inherited authorization	ARK-452
Auditability: independent verifier detects all tested tamper attempts	ARK-455

ARK-455 would be the first experiment in the corpus to directly validate the **commercial ProofRecord artifact** that external auditors would inspect. It connects the research program directly to the independently verifiable record that enterprise buyers demand.

15.2 What ARK-455 Enables Downstream

- **ARK-456 (Dependency Loss)** becomes stronger with ARK-455 in the corpus. If a required external dependency (signature key, schema registry, audit store) is unavailable, the system should fail closed. ARK-455 establishes the baseline verification logic; ARK-456 tests its behavior under degraded infrastructure.
- **ARK-450 (Substitution Attacks)** can reference ARK-455's verification model. If an attacker substitutes one ProofRecord for another (not tampering a single field, but swapping entire records), does the verifier detect the mismatch? ARK-455 establishes single-field detection; ARK-450 extends to record-level substitution.
- **Commercial deployment:** ARK-455 provides the independently reproducible verification algorithm that external auditors, regulators, and enterprise compliance teams can implement in their own tooling.

15.3 Items Explicitly Deferred from ARK-455

- **Multi-field tampering.** Altering multiple fields simultaneously is deferred. ARK-455 tests single-field detection as the baseline.
- **Signature forgery.** Attempting to generate a valid signature for a tampered record without the private key is deferred to a future cryptographic-focused experiment.

- **Schema evolution.** What happens when the ProofRecord schema changes and old records must be verified under a new schema version is deferred.
- **Performance / scale.** Verification throughput, latency, and behavior on millions of records are deferred. ARK-455 tests correctness on 800 records total.

16. MANIFEST Placeholder

Compute and fill at lock time, immediately before kill-gate execution. Do not fill in advance.

```
ARK-455 MANIFEST – Version 1.0
Lock timestamp: [FILL AT LOCK – UTC, format YYYY-MM-DDTHH:MM:SSZ]
Execution substrate: Local classical software (no IBM QPU)

SHA-256 hashes:
  PREREGISTRATION.md: [COMPUTE AT LOCK]
  schemas/proofrecord_schema.json: [COMPUTE AT LOCK]
  verifiers/v1_verifier.ts: [COMPUTE AT LOCK]
  verifiers/v2_verifier.py: [COMPUTE AT LOCK]
  generator/record_generator.py: [COMPUTE AT LOCK]

MANIFEST SHA-256 (of this file after all hashes filled):
  [COMPUTE AT LOCK]

Committed to: executionproof-testbeds
Branch:      main (or execute/ark-455, per repository convention)
Tag:         ark-455-v1.0-lock
Commit SHA:  [FILL AFTER PUSH]

Kill-gate outcome: [FILL AFTER CALIBRATION]
Arm execution start: [FILL AT ARM 1 START]
```

17. ProofRecord Stub

A `proofrecord.json` will be generated at the conclusion of the experiment, following the format established in ARK-448 and ARK-449. It will contain:

```

{
  "experiment": "ARK-455",
  "doctrine_tested": "An independent verifier can detect every tampering attempt and
reject altered records",
  "verdict": "[PASS | FAIL | KILLED AT CALIBRATION GATE]",
  "timestamp_lock": "[UTC]",
  "timestamp_execution": "[UTC]",
  "substrate": "classical software (local)",
  "verifiers": {"V1": "TypeScript", "V2": "Python"},
  "kill_gate": {
    "concordance_pct": null,
    "sanity_valid_accept_pct": null,
    "passed": null
  },
  "primary_metrics": {
    "V_accept_original": null,
    "V_reject_min": null,
    "V_margin": null
  },
  "secondary_metrics": {
    "V_reject_per_arm": {},
    "concordance_per_arm": {}
  },
  "criteria": {
    "C1_V_accept_pass": null,
    "C2_V_reject_pass": null,
    "C3_V_margin_pass": null
  },
  "manifest_sha256": "[FILL AT LOCK]",
  "zenodo_doi": "[FILL AT PUBLICATION]"
}

```

This document is the preregistration for ARK-455. It is locked at the time of MANIFEST commit and tag. No changes to hypotheses, criteria, arms, thresholds, or analysis plan are permitted after that point. A FAIL or KILLED is a valid, publishable result that advances the program. The value of the protocol is precisely that it constrains choices before any data is seen and keeps the record whole regardless of outcome.

Remnant Fieldworks Inc. — ExecutionProof ARK Series
<https://github.com/derekhone/executionproof-testbeds>